

Candidate Name : Jagruti Ramesh Mali

Role Applied : Backend Engineering Intern

Submission Format : Case Study with Explanation and code

Inventory Management System for B2B SaaS – StockFlow

Overview

StockFlow is a B2B inventory management system designed for small businesses to monitor product levels across multiple warehouses and manage supplier relationships efficiently. It supports complex product structures (including bundles), warehouse-based quantity tracking, and real-time stock alerts. The system enables smarter purchasing decisions, streamlined inventory operations, and reduces the risk of stockouts by providing actionable alerts.

Tools & Technologies Used

- **Python** – For backend logic and API development
 - **Flask** – Lightweight web framework to build REST API
 - **SQLite** – Used as the database for development and testing
 - **SQLAlchemy** – ORM to manage database models and queries
 - **Visual Studio Code (VS Code)** – Code editor used for the entire project
 - **Postman** – For testing API endpoints (GET/POST)
-

Part 1: Code Review & Debugging

In this task, I reviewed a Flask API endpoint for adding new products to the system. I identified several issues, including missing input validation, no SKU uniqueness check, lack of error handling, and multiple database commits. These problems could lead to data inconsistency and app crashes in production. I then corrected the code to ensure proper validation, atomic transactions, and a clear JSON response.

No.	Issue Type	Description of Issue	Impact in Production	Fix Implemented
1	Technical	No validation of JSON input	Application will throw TypeError or KeyError on empty or malformed JSON body	Added if not data check with 400 error response
2	Business Logic	No check for missing fields	Missing fields can cause runtime errors or incomplete product data	Added a loop to validate all required fields
3	Technical	No data type validation for price, quantity, and warehouse_id	Can cause exceptions or incorrect database values	Added explicit conversion using Decimal() and int() with ValueError handling
4	Business Logic	Price or quantity can be negative	Leads to invalid business records (e.g., negative inventory)	Added checks: return 400 error if price/quantity < 0
5	Business Logic	No check for warehouse existence	Inventory may be created with a nonexistent warehouse (foreign key issues)	Checked if warehouse exists; return 404 if not found
6	Business Logic	SKU uniqueness not enforced in logic	Database will throw IntegrityError on duplicate SKUs	Checked for existing SKU before insert; return 409 Conflict
7	Technical	Two-stage DB operations not in transaction	Can cause partial insert if second fails (product saved but inventory not)	Used db.session.begin_nested() and flush() before commit
8	Business Logic	Duplicate product-warehouse pair not reused	May create duplicate inventory if uniqueness constraint fails	Reused inventory if it exists, otherwise created new

9	Technical	No proper exception handling	Any error would crash the app or give generic 500 error	Added exception handling for ValueError, IntegrityError, OperationalError, and generic errors
10	Technical	No logging for unexpected errors	Debugging issues in production would be hard	Used app.logger.error() with traceback for debugging
11	Business Logic	create_all() used directly	Not a scalable approach in production	Included note to use migrations (e.g., Alembic) in production

Fixed Issue Code

```

from flask import Flask, request, jsonify, Blueprint # Import Blueprint
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.exc import IntegrityError, OperationalError
from decimal import Decimal
from datetime import datetime # Needed for timestamps in models

# --- Application Initialization (Typically in app/__init__.py) ---
# This part sets up the Flask app and database.
# For the report, it's included here for completeness of the "Fixed Code".
app = Flask(__name__)

# Configure SQLAlchemy database (using SQLite for local testing)
# This should match your config.py setting
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///inventory.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False # Suppresses a warning
db = SQLAlchemy(app)

# Create a Blueprint instance for API routes.
# All routes defined within this blueprint will automatically be prefixed with
# '/api'.
api_bp = Blueprint('api_bp', __name__, url_prefix='/api')

# --- Database Models (Typically in app/models.py) ---
# These models define the database schema and relationships.

class Company(db.Model):
    __tablename__ = 'companies'
    company_id = db.Column(db.Integer, primary_key=True)
    company_name = db.Column(db.String(255), nullable=False)
    # Added timestamps for better tracking
    created_at = db.Column(db.DateTime, default=datetime.now)

```

```

        updated_at = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)

class Warehouse(db.Model):
    __tablename__ = 'warehouses'
    warehouse_id = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('companies.company_id'),
nullable=False)
    warehouse_name = db.Column(db.String(255), nullable=False)
    location = db.Column(db.Text)
    # Added timestamps
    created_at = db.Column(db.DateTime, default=datetime.now)
    updated_at = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)
    company = db.relationship('Company', backref='warehouses')

class ProductType(db.Model):
    __tablename__ = 'product_types'
    product_type_id = db.Column(db.Integer, primary_key=True)
    type_name = db.Column(db.String(100), nullable=False, unique=True)
    default_low_stock_threshold = db.Column(db.Integer, default=10)
    # Added timestamps
    created_at = db.Column(db.DateTime, default=datetime.now)
    updated_at = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)

class Product(db.Model):
    __tablename__ = 'products'
    # FIX: Changed primary key name from 'id' to 'product_id' to match foreign
key references
    product_id = db.Column(db.Integer, primary_key=True)
    # FIX: Changed column name from 'name' to 'product_name' to match model
property
    product_name = db.Column(db.String(255), nullable=False)
    sku = db.Column(db.String(255), unique=True, nullable=False) # Unique
identifier
    price = db.Column(db.Numeric(10, 2), nullable=False) # Decimal
format for precision
    product_type_id = db.Column(db.Integer,
db.ForeignKey('product_types.product_type_id'))
    # Added timestamps
    created_at = db.Column(db.DateTime, default=datetime.now)
    updated_at = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)
    product_type = db.relationship('ProductType', backref='products')

class Inventory(db.Model):
    __tablename__ = 'inventory'

```

```

    # FIX: Foreign key now correctly references 'products.product_id'
    product_id = db.Column(db.Integer, db.ForeignKey('products.product_id'),
primary_key=True)
    # FIX: Foreign key now correctly references 'warehouses.warehouse_id'
    warehouse_id = db.Column(db.Integer,
db.ForeignKey('warehouses.warehouse_id'), primary_key=True)
    quantity = db.Column(db.Integer, nullable=False, default=0)
    last_updated = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)
    low_stock_threshold = db.Column(db.Integer) # Per-inventory override for
threshold

    # ORM relationships for easier access
    product = db.relationship('Product', backref='inventory_records')
    warehouse = db.relationship('Warehouse', backref='inventory_records')

    # Ensure each product can exist only once per warehouse
    __table_args__ = (db.UniqueConstraint('product_id', 'warehouse_id',
name='_product_warehouse_uc'),)

# Note: Other models (Supplier, ProductSupplier, SalesActivity, Bundles,
BundleItems)
# would also be defined here as per your Part 2 schema, but are omitted for
brevity
# in this specific "Fixed Issue Code" focused on the product creation
endpoint.

# --- API Endpoint: Create Product (Typically in app/routes.py) ---
# This is the corrected Flask route for adding new products.

@api_bp.route('/products', methods=['POST']) # FIX: Using blueprint route
decorator
def create_product():
    data = request.json

    # FIX: Step 1: Check if request body is valid JSON
    if not data:
        return jsonify({"message": "Invalid JSON data"}), 400

    # FIX: Step 2: Check for missing required fields
    required_fields = ['name', 'sku', 'price', 'warehouse_id',
'initial_quantity']
    for field in required_fields:
        if field not in data:
            return jsonify({"message": f"Missing required field: {field}"}),
400

    try:

```

```

# FIX: Step 3: Type conversion and validation
name = data['name']
sku = data['sku']
price = Decimal(str(data['price'])) # Ensure decimal precision
warehouse_id = int(data['warehouse_id'])
initial_quantity = int(data['initial_quantity'])

# FIX: Step 4: Validate positive values
if price < 0:
    return jsonify({"message": "Price cannot be negative"}), 400
if initial_quantity < 0:
    return jsonify({"message": "Initial quantity cannot be
negative"}), 400

# FIX: Step 5: Ensure warehouse exists
warehouse = Warehouse.query.get(warehouse_id)
if not warehouse:
    return jsonify({"message": f"Warehouse with ID {warehouse_id} not
found"}), 404

# FIX: Step 6: Ensure SKU is unique (handle conflict early)
existing_product = Product.query.filter_by(sku=sku).first()
if existing_product:
    return jsonify({"message": f"Product with SKU '{sku}' already
exists."}), 409

# FIX: Step 7: Start DB transaction for atomicity
db.session.begin_nested()

# FIX: Step 8: Create product instance with correct keyword argument
product = Product(product_name=name, sku=sku, price=price)
db.session.add(product)
db.session.flush() # Assigns product.product_id

# FIX: Step 9: Update inventory - either update existing or create new
record
inventory = Inventory.query.filter_by(
    product_id=product.product_id, # FIX: Use product.product_id
    warehouse_id=warehouse_id
).first()

if inventory:
    inventory.quantity += initial_quantity
else:
    inventory = Inventory(
        product_id=product.product_id, # FIX: Use product.product_id
        warehouse_id=warehouse_id,
        quantity=initial_quantity

```

```

    )
    db.session.add(inventory)

    # FIX: Step 10: Final commit for both product and inventory
    db.session.commit()

    # FIX: Step 11: Return successful response with correct product ID
    return jsonify({"message": "Product created successfully",
"product_id": product.product_id}), 201

# FIX: Comprehensive Exception Handling Section
except ValueError as e:
    db.session.rollback()
    return jsonify({"message": f"Invalid data type or value: {e}"}), 400
except IntegrityError as e:
    db.session.rollback()
    return jsonify({"message": f"Database integrity error: {e.orig.args[0]
if e.orig else str(e)}"}), 400
except OperationalError as e:
    db.session.rollback()
    return jsonify({"message": f"Database operational error: {e}"}), 500
except Exception as e:
    db.session.rollback()
    # FIX: Log unexpected errors for investigation
    app.logger.error(f"An unexpected error occurred: {e}", exc_info=True)
    return jsonify({"message": "An unexpected error occurred"}), 500

# --- Application Run (Typically in run.py) ---
# This part is for running the Flask development server.
# In a real setup, db.create_all() and seeding would be handled by a CLI
command or migrations.

# Register the blueprint with the app (this would be in app/__init__.py in the
final structure)
app.register_blueprint(api_bp)

# Create all tables (only for testing/dev - in production use migrations like
Alembic)
# This part would typically be in run.py, inside an app.app_context() block.
with app.app_context():
    db.create_all()
    # Note: Data seeding logic would also be here, as shown in your run.py
file.
    # It's omitted here for brevity, as this section focuses on the endpoint
fix.

if __name__ == '__main__':
    app.run(debug=True) # Set debug=False in production

```

Part 2: Database Design

In this part, I designed a database schema for a B2B inventory management system. I understood that the system is meant to help companies manage their products across multiple warehouses. Each product can be stored in different warehouses with separate quantities. We also need to track changes in inventory levels, manage relationships with suppliers, and allow certain products to be grouped together as bundles.

The goal was to build a clean, well-structured, and scalable schema that ensures data accuracy, handles different business rules, and supports possible future features like alerts, sales tracking, and reporting.

What I did :

- Created a Companies table with company ID and name.
- Created a Warehouses table linked to companies using company ID.
- Created a ProductTypes table for different product types and default thresholds.
- Created a Products table with unique SKU, price, and type reference.
- Created an Inventory table with product ID, warehouse ID, and quantity.
- Added last_updated and low_stock_threshold in Inventory table.
- Created an InventoryHistory table to log every inventory change.
- Created a Suppliers table with basic contact details.
- Created a ProductSuppliers table to map products and suppliers.
- Created a Bundles table to store bundle product details.
- Created a BundleItems table to link products inside bundles.
- Created a SalesActivity table to track product sales in warehouses.
- Used primary keys, foreign keys, and unique constraints for accuracy.
- Used timestamps for created and updated time in all main tables.
- Used CHECK constraints to ensure valid quantity and sales data.
- Designed schema to allow tracking, reporting, and alerting features.
- Noted down missing details like user roles, return handling, batch tracking, etc.

1. Design Schema (SQL DDL Notation):

-- Design Schema (SQL DDL Notation):

-- Table to store company details (each company using the system)

```
CREATE TABLE Companies (  
    company_id INT PRIMARY KEY AUTO_INCREMENT,  
    company_name VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- Record creation time  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
-- Last updated time  
);
```

-- Table to store warehouses linked to specific companies


```

CREATE TABLE Warehouses (
    warehouse_id INT PRIMARY KEY AUTO_INCREMENT,
    company_id INT NOT NULL, -- Each warehouse belongs to a company
    warehouse_name VARCHAR(255) NOT NULL,
    location TEXT, -- Address or location of the warehouse
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (company_id) REFERENCES Companies(company_id) ON DELETE
CASCADE -- Delete warehouses if company is deleted
);

-- Table to define different product types with default low-stock threshold
CREATE TABLE ProductTypes (
    product_type_id INT PRIMARY KEY AUTO_INCREMENT,
    type_name VARCHAR(100) NOT NULL UNIQUE, -- e.g., Electronics, Groceries
    default_low_stock_threshold INT DEFAULT 10, -- Used to trigger low-stock
alerts
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Table to store product details
CREATE TABLE Products (
    product_id INT PRIMARY KEY AUTO_INCREMENT, -- Corrected: Primary key named
product_id
    product_name VARCHAR(255) NOT NULL, -- Corrected: Column named
product_name
    sku VARCHAR(255) NOT NULL UNIQUE, -- Unique product code across platform
description TEXT,
    price DECIMAL(10, 2) NOT NULL, -- Product price with decimals
    product_type_id INT, -- Links to product category/type
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (product_type_id) REFERENCES ProductTypes(product_type_id) ON
DELETE SET NULL -- If type deleted, keep product
);

-- Table to manage quantity of products in each warehouse
CREATE TABLE Inventory (
    product_id INT,
    warehouse_id INT,
    quantity INT NOT NULL DEFAULT 0 CHECK (quantity >= 0), -- Stock amount
must be 0 or more
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP, -- Last stock update

```

```

        low_stock_threshold INT, -- Optional custom threshold per product-
warehouse
        PRIMARY KEY (product_id, warehouse_id), -- Unique product entry per
warehouse
        FOREIGN KEY (product_id) REFERENCES Products(product_id) ON DELETE
CASCADE, -- Corrected FK reference
        FOREIGN KEY (warehouse_id) REFERENCES Warehouses(warehouse_id) ON DELETE
CASCADE
    );

-- Table to log every change in inventory (e.g., sale, return, adjustment)
CREATE TABLE InventoryHistory (
    history_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT NOT NULL,
    warehouse_id INT NOT NULL,
    change_type ENUM('inbound', 'outbound', 'adjustment') NOT NULL, -- Reason
for change
    quantity_change INT NOT NULL, -- +ve or -ve change amount
    new_quantity INT NOT NULL, -- Final quantity after change
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    notes TEXT, -- Optional comment or reason
    FOREIGN KEY (product_id, warehouse_id) REFERENCES Inventory(product_id,
warehouse_id) ON DELETE CASCADE
);

-- Table to store supplier information
CREATE TABLE Suppliers (
    supplier_id INT PRIMARY KEY AUTO_INCREMENT,
    supplier_name VARCHAR(255) NOT NULL,
    contact_person VARCHAR(255),
    contact_email VARCHAR(255),
    phone_number VARCHAR(50),
    address TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Table to map which supplier provides which product (many-to-many)
CREATE TABLE ProductSuppliers (
    product_id INT,
    supplier_id INT,
    PRIMARY KEY (product_id, supplier_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id) ON DELETE
CASCADE, -- Corrected FK reference
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id) ON DELETE
CASCADE
);

```

```

-- Table to store bundle product information
CREATE TABLE Bundles (
    bundle_id INT PRIMARY KEY AUTO_INCREMENT,
    bundle_name VARCHAR(255) NOT NULL,
    sku VARCHAR(255) NOT NULL UNIQUE, -- Unique bundle code
    description TEXT,
    price DECIMAL(10, 2), -- Bundle price can be fixed or calculated
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Table to store which products are included in a bundle and their quantity
CREATE TABLE BundleItems (
    bundle_item_id INT PRIMARY KEY AUTO_INCREMENT,
    bundle_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0), -- Quantity per product in
bundle
    FOREIGN KEY (bundle_id) REFERENCES Bundles(bundle_id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES Products(product_id) ON DELETE
CASCADE, -- Corrected FK reference
    UNIQUE (bundle_id, product_id) -- Avoid duplicate entries for the same
product in a bundle
);

-- Table to store sales data for products to help with stock analysis
CREATE TABLE SalesActivity (
    sale_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT NOT NULL,
    warehouse_id INT NOT NULL,
    company_id INT NOT NULL,
    quantity_sold INT NOT NULL CHECK (quantity_sold > 0), -- Positive number
of items sold
    sale_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id, warehouse_id) REFERENCES Inventory(product_id,
warehouse_id), -- Directly links sold inventory
    FOREIGN KEY (company_id) REFERENCES Companies(company_id) ON DELETE
CASCADE
);

```

Part 3: API Implementation - Low Stock Alerts

I learned that this feature helps companies keep track of products that are running low in their warehouses. The system checks if a product's stock has fallen below a certain limit (called a threshold) and whether there have been any recent sales of that product in the last 30 days. If both conditions are true, it sends an alert. The alert also shows how many days the stock might last based on how fast the product is selling. It even includes supplier details so the company can reorder quickly. This makes it easier for businesses to avoid running out of products and manage their inventory more efficiently.

What I did :

- Built a GET API to return low stock alerts for a company.
- Verified company existence using `company_id`.
- Fetched inventory and joined related product, warehouse, and product type data.
- Checked stock levels against dynamic thresholds.
- Filtered alerts only for products with recent sales (last 30 days).
- Calculated `days_until_stockout` based on average daily sales.
- Retrieved all linked supplier details for each product.
- Returned alerts in the required JSON format.
- Handled edge cases like no company, no sales, zero stock, or missing data.

Implementation :

```
from flask import Flask, request, jsonify, Blueprint # Import Blueprint
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.exc import IntegrityError, OperationalError # Included for
completeness, though not strictly used in GET
from sqlalchemy import func, distinct
from datetime import datetime, timedelta
from decimal import Decimal

# --- Application Initialization (Simplified for report context) ---
# In a full application, this would be handled by app/__init__.py and run.py.
# For the report, it's included here to make the code self-contained.

app = Flask(__name__)
# Use a persistent SQLite for demonstration in report (matches your run.py)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///inventory.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

# Create a Blueprint instance for API routes.
api_bp = Blueprint('api_bp', __name__, url_prefix='/api')

# --- Database Models (Relevant to Low Stock Alerts) ---
```

```
# These are the models directly involved in the low stock alert logic.
# All primary keys and foreign keys are consistent with the final schema.
```

```
class Company(db.Model):
    __tablename__ = 'companies'
    company_id = db.Column(db.Integer, primary_key=True)
    company_name = db.Column(db.String(255), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.now)
    updated_at = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)

class Warehouse(db.Model):
    __tablename__ = 'warehouses'
    warehouse_id = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('companies.company_id'),
nullable=False)
    warehouse_name = db.Column(db.String(255), nullable=False)
    location = db.Column(db.Text)
    created_at = db.Column(db.DateTime, default=datetime.now)
    updated_at = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)
    company = db.relationship('Company', backref='warehouses')

class ProductType(db.Model):
    __tablename__ = 'product_types'
    product_type_id = db.Column(db.Integer, primary_key=True)
    type_name = db.Column(db.String(100), nullable=False, unique=True)
    default_low_stock_threshold = db.Column(db.Integer, default=10)
    created_at = db.Column(db.DateTime, default=datetime.now)
    updated_at = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)

class Product(db.Model):
    __tablename__ = 'products'
    product_id = db.Column(db.Integer, primary_key=True)
    product_name = db.Column(db.String(255), nullable=False)
    sku = db.Column(db.String(255), nullable=False, unique=True)
    description = db.Column(db.Text)
    price = db.Column(db.Numeric(10, 2), nullable=False)
    product_type_id = db.Column(db.Integer,
db.ForeignKey('product_types.product_type_id'))
    created_at = db.Column(db.DateTime, default=datetime.now)
    updated_at = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)
    product_type = db.relationship('ProductType', backref='products')

class Inventory(db.Model):
    __tablename__ = 'inventory'
```

```

    product_id = db.Column(db.Integer, db.ForeignKey('products.product_id'),
primary_key=True)
    warehouse_id = db.Column(db.Integer,
db.ForeignKey('warehouses.warehouse_id'), primary_key=True)
    quantity = db.Column(db.Integer, nullable=False, default=0)
    last_updated = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)
    low_stock_threshold = db.Column(db.Integer) # Per-inventory override for
threshold

```

```

product = db.relationship('Product', backref='inventory_records')
warehouse = db.relationship('Warehouse', backref='inventory_records')

```

```

class Supplier(db.Model):
    __tablename__ = 'suppliers'
    supplier_id = db.Column(db.Integer, primary_key=True)
    supplier_name = db.Column(db.String(255), nullable=False)
    contact_person = db.Column(db.String(255))
    contact_email = db.Column(db.String(255))
    phone_number = db.Column(db.String(50))
    created_at = db.Column(db.DateTime, default=datetime.now)
    updated_at = db.Column(db.DateTime, default=datetime.now,
onupdate=datetime.now)

```

```

class ProductSupplier(db.Model):
    __tablename__ = 'product_suppliers'
    product_id = db.Column(db.Integer, db.ForeignKey('products.product_id'),
primary_key=True)
    supplier_id = db.Column(db.Integer,
db.ForeignKey('suppliers.supplier_id'), primary_key=True)
    supplier = db.relationship('Supplier', backref='product_associations')
    product = db.relationship('Product', backref='supplier_associations')

```

```

class SalesActivity(db.Model):
    __tablename__ = 'sales_activity'
    sale_id = db.Column(db.Integer, primary_key=True)
    product_id = db.Column(db.Integer, db.ForeignKey('products.product_id'),
nullable=False)
    warehouse_id = db.Column(db.Integer,
db.ForeignKey('warehouses.warehouse_id'), nullable=False)
    company_id = db.Column(db.Integer, db.ForeignKey('companies.company_id'),
nullable=False)
    quantity_sold = db.Column(db.Integer, nullable=False)
    sale_date = db.Column(db.DateTime, default=datetime.now)

    product = db.relationship('Product', backref='sales')
    warehouse = db.relationship('Warehouse', backref='sales')
    company = db.relationship('Company', backref='sales')

```

```

# --- API Endpoint: Low Stock Alerts ---
# This is the Flask route for fetching low stock alerts.

@api_bp.route('/companies/<int:company_id>/alerts/low-stock', methods=['GET'])
def get_low_stock_alerts(company_id):
    """
    Returns low-stock alerts for a specific company, considering product type
    thresholds,
    per-inventory overrides, and recent sales activity.
    """
    try:
        # 1. Validate company_id
        company = Company.query.get(company_id)
        if not company:
            return jsonify({"message": f"Company with ID {company_id} not
found"}), 404

        alerts = []
        # Define the window for "recent sales activity" (e.g., last 30 days)
        thirty_days_ago = datetime.now() - timedelta(days=30)

        # Query for all inventory items associated with the company's
        warehouses
        inventory_items = db.session.query(
            Inventory, Product, ProductType, Warehouse
        ).join(
            Product, Inventory.product_id == Product.product_id
        ).join(
            Warehouse, Inventory.warehouse_id == Warehouse.warehouse_id
        ).outerjoin( # Use outerjoin for ProductType as it might be NULL if ON
DELETE SET NULL was triggered
            ProductType, Product.product_type_id ==
ProductType.product_type_id
        ).filter(
            Warehouse.company_id == company_id
        ).all()

        for inventory, product, product_type, warehouse in inventory_items:
            # Determine effective low stock threshold
            threshold = inventory.low_stock_threshold
            if threshold is None:
                if product_type:
                    threshold = product_type.default_low_stock_threshold
                else:
                    threshold = 10 # Fallback if no product type or override
            is set

```

```

        if not isinstance(threshold, int): # Defensive check
            threshold = 10

        # Check if current stock is below the threshold
        if inventory.quantity <= threshold:
            # Check for recent sales activity for this specific product in
this warehouse
            recent_sales_sum =
db.session.query(func.sum(SalesActivity.quantity_sold)).filter(
                SalesActivity.product_id == product.product_id,
                SalesActivity.warehouse_id == warehouse.warehouse_id,
                SalesActivity.sale_date >= thirty_days_ago
            ).scalar() or 0

            # Only alert if there's recent sales activity (sum of sales >
0)

            if recent_sales_sum > 0:
                days_until_stockout = None
                earliest_recent_sale_date =
db.session.query(func.min(SalesActivity.sale_date)).filter(
                    SalesActivity.product_id == product.product_id,
                    SalesActivity.warehouse_id == warehouse.warehouse_id,
                    SalesActivity.sale_date >= thirty_days_ago
                ).scalar()

                if earliest_recent_sale_date:
                    num_days_sales_tracked = (datetime.now() -
earliest_recent_sale_date).days
                    if num_days_sales_tracked == 0:
                        num_days_sales_tracked = 1
                    else:
                        num_days_sales_tracked = 1

                    average_daily_sales = recent_sales_sum / max(1,
num_days_sales_tracked)

                    if average_daily_sales > 0:
                        days_until_stockout = round(inventory.quantity /
average_daily_sales)

                # Get supplier information
                suppliers_data = []
                product_suppliers = db.session.query(ProductSupplier,
Supplier).join(
                    Supplier, ProductSupplier.supplier_id ==
Supplier.supplier_id
                ).filter(
                    ProductSupplier.product_id == product.product_id

```



```

    ).all()

    for ps, supplier in product_suppliers:
        suppliers_data.append({
            "id": supplier.supplier_id,
            "name": supplier.supplier_name,
            "contact_email": supplier.contact_email
        })

    alerts.append({
        "product_id": product.product_id,
        "product_name": product.product_name,
        "sku": product.sku,
        "warehouse_id": warehouse.warehouse_id,
        "warehouse_name": warehouse.warehouse_name,
        "current_stock": inventory.quantity,
        "threshold": threshold,
        "days_until_stockout": days_until_stockout,
        "supplier": suppliers_data
    })

    return jsonify({
        "alerts": alerts,
        "total_alerts": len(alerts)
    }), 200

except Exception as e:
    # Log the error for debugging
    app.logger.error(f"Error fetching low stock alerts for company {company_id}: {e}", exc_info=True)
    return jsonify({"message": "An unexpected error occurred while fetching alerts"}), 500

# --- Database Creation and App Run (Simplified for report context) ---
# In a full application, this would be handled by run.py.
# For the report, it's included here to make the code self-contained for demonstration.

# Register the blueprint with the app
app.register_blueprint(api_bp)

# Create all tables (only for testing/dev - in production use migrations)
with app.app_context():
    db.create_all()

if __name__ == '__main__':
    app.run(debug=True) # Set debug=False in production

```

Conclusion :

Working on the "StockFlow" B2B inventory system gave me a real-world experience of how full-stack applications are built and improved. I learned how to identify and fix problems in existing code, especially issues related to input checks, database errors, and safe transactions. I made the product creation API more reliable and followed key business rules like unique SKUs and correct inventory updates.

Designing the database helped me understand how to create strong relationships between tables, like Products in multiple Warehouses and multiple Suppliers for a product. I also learned the importance of asking the right questions when details are missing.

Finally, building the low-stock alerts API helped me apply everything together — from logic like dynamic stock thresholds to calculating how long stock will last based on recent sales. It improved my API development skills and showed me how to think from both a technical and business point of view. This case study made me more confident in full-stack development, problem solving, and handling real project challenges.

Reference :

1. Wikipedia contributors. (2024, May 12). *Inventory management software*. Wikipedia. https://en.wikipedia.org/wiki/Inventory_management_software
2. Wikipedia contributors. (2024, April 25). *Inventory management (business)*. Wikipedia. [https://en.wikipedia.org/wiki/Inventory_management_\(business\)](https://en.wikipedia.org/wiki/Inventory_management_(business))
3. Stack Overflow contributors. (2020, May 6). *Designing an item stock alert architecture*. Stack Overflow. <https://stackoverflow.com/questions/61599220/designing-an-item-stock-alert-architecture>
4. Stack Overflow contributors. (2018, Dec 5). *How to handle quantity/stock monitoring in an inventory system*. Stack Overflow. <https://stackoverflow.com/questions/53650109/how-to-handle-quantity-stock-monitoring-in-an-inventory-system>